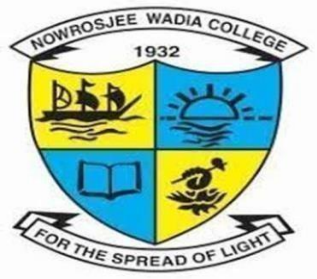
Modern Education Society's

Nowrosjee Wadia College, Pune

An Autonomous college affiliated to Savitribai Phule Pune University



Department of Computer Science

T.Y.B.Sc(Computer Science) NEP 1.0

Choice Based Credit System to be implemented from Academic Year 2025-2026

WORKBOOK

CSMJ364

Laboratory Course Based on

CSMJ361 (Operating System-II and Project Implementation)

SEMESTER VI

Student Name:			
Roll No:		**Division:**	
Year:		**Exam Seat No:**	

Table of contents

Introduction

About the workbook

This workbook is intended to be used by T. Y. B. Sc (Computer Science) students for the Practical Course based on CSMJ361 (Operating Systems-II) in Semester VI.

Operating System is an important core subject of computer science curriculum, and hands-on laboratory experience is critical to the understanding of theoretical concepts studied as part of this course. Study of any programming language is incomplete without hands on experience of implementing solutions using programming paradigms and verifying them in the lab. This workbook provides rich set of problems covering the basic algorithms as well as numerous computing problems demonstrating the applicability and importance of various data structures and related algorithms.

About the Work Book Objectives –

1. Defining clearly the scope of the course.
2. Continuous assessment of the students.
3. Bring variation and variety in experiments carried out by different students in a batch
4. Providing ready references for students while working in the lab.
5. Catering to the need of slow paced as well as fast paced learners

How to use this book?

This book is mandatory for the completion of the laboratory course. It is a measure of the performance of the student in the laboratory for the entire duration of the course.

The Operating Systems-I practical syllabus is divided into four assignments. Each assignment has problems divided into Slots.

Instructions to the students:

Please read the following instructions carefully and follow them

- Students are expected to carry workbook during every practical.
- Students should prepare oneself beforehand for the Assignment by reading the relevant material.
- Teacher/Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor. However, student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.
- **Students will be assessed for each exercise on a scale from 0 to 5**

Not done	0
Incomplete	1
Late Complete	2
Needs improvement	3
Complete	4
Well Done	5

Instruction to the Practical In-Charge:

- Explain the assignment and related concepts in around ten minutesusing white board if required or by demonstrating the software.
- Choose appropriate problems to be solved by students. Slot I is mandatory. Choose problems from Slot II depending on time availability. Discuss Slot III with students and encourage them to solve the problems by spending additional time in lab or at home.
- Make sure that students follow the instruction as given above.
- You should evaluate each assignment carried out by a student on a scale of 5 as specified above by ticking appropriate box.
- The value should also be entered on assignment completion page of the respective Lab course.

Instructions to the Lab administrator and Exam guidelines:

- You have to ensure appropriate hardware and software is made available to each student
- Do not provide Internet facility in Computer Lab while examination
- Do not provide pen drive facility in Computer Lab while examination.
- The operating system and software requirements are as given below:
- Operating system: Linux
- Editor: Any Linux based editor like vi, gedit etc
- Compiler: cc or gcc

Assignment Completion Sheet

Sr. No.		Assignment Name	Marks (out of 5)	Signature
a	1.	Simulation of Banker's Algorithm of Deadlock Avoidance in processes of operating system		
	2.	Simulation of File Allocation Methods and free space management		
	3.	Simulation of Disk Scheduling Algorithms		
	4.	Assignment based on Mobile OS		
	5.	Project Demo1(Out of 5)		
	6.	Project Demo2(Out of 5)		
	7.	Project Demo3(Out of 10)		
	8.	Project Report(Out of 5)		
Total out of 45				
b. Conduct Viva at the time of Submission				
a. Total out of 10				
b. Total out of 5				
c. Total(Out of 15) (a + b)				

This is to certify that Mr. /Ms _____

Exam Seat Number_____has successfully completed the coursework for Lab Course I and has scored _____Marks out of 15.

Head

Teacher/Instructor In charge

Dept. of Computer Science

Assignment No. 1:

Banker's Algorithm

Introduction: This algorithm is related to handling deadlocks. There are three methods to handle deadlocks.

1. Deadlock Prevention
2. Deadlock Avoidance
3. Deadlock Detection

Banker's algorithm is a technique used for **Deadlock Avoidance**. A deadlock avoidance algorithm dynamically examines the resource allocation state of system to the processes to ensure that a **circular-wait condition never exists.** The resource allocation state is defined by the number of available and allocated resources and the maximum demands of the processes. A system is **safe** if the system can allocate resources to each process in some order and still avoid a deadlock. i.e. a system is in a safe state only if there exists a **safe sequence**. A sequence of processes <P1, P2, …., P_n> is a safe sequence for the current allocation state if, for each P_i , the resources that P_i can still request can be satisfied by currently available resources plus the resources held by all the Pj, with j < i.

Resource allocation graph is a technique used for deadlock avoidance. But the resource allocation graph algorithm is not applicable to the system with multiple instances of each resource type. Banker's algorithm is applicable to such a system.

Banker's algorithm: When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system. When a user requests a set of resources, the system must determine whether the allocation of these resources will keep the system in a safe state. If it will, then resources are allocated, otherwise the process must wait until some other process releases enough resources. This includes 2 algorithms

1) **Safety Algorithm**
2) **Resource-Request Algorithm**

Data Structure Design:

Let n be the number of processes in the system and m be the number of resource types.

Component	Description	'c' code
Available	A vector of length m indicates the number of available resources of each type	int Avail[10]
Max	An n x m matrix which defines the maximum demand of each process	int Max[10][10]
Allocation	An n x m matrix that defines the number of resources of each type currently allocated to each process	int Alloc[10[10]
Need	An n x m matrix indicates the remaining resource need of each process.	int Need[10][10]
Request	A vector of length m indicates the request made by a process Pi	int Request[10]
Finish	Vector of length n to check whether the process finished or not	int Finish[10] ={0}
Work	Vector of length m which is initialized to Available	int Work[10]
Safe	Vector of length n initialized to 0	Int Safe[10]= {0};

Safety Algorithm: It is a safety algorithm used to check whether or not a system is in a safe state or follows the safe sequence in a banker's algorithm:

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively.
2. Initialize:

   Work = Available

   Finish [*i*] = *0 for i* = 1,2,3, …, *n.*

   SafeSequence[i] =0 for i = 1,2,3, …, *n.*
3. Find an *i* such that both:

   (a) *Finish* [*i*] = 0

   (b) $Need_i = Work$

   (c) Add *Pi* in the array of *SafeSequence*

   If no such *i* exists, go to step 4.
4. *Work=Work+Allocation*$_i$
5. *Finish*[*i*]=1

   go to step 2.

6. If *Finish* [*i*] == 1 for all *i*, then the system is in a safe state.

Resource-Request Algorithm for Process *Pi:*

Once the system is in safe state, a Resource-Request algorithm is used for determining whether the request by a process can be satisfied or not.

Let *Requesti* be a request vector for the process *Pi*. Process *Pi* request *k* instances of resource type *Rj*, which is indicated by *Request [j] = k.*
The following things happen when process *Pi* makes a request for resources:

a) Proceed to step b **if** *Requesti* <= *Needi*; otherwise, report an error condition because the process has made more claims than it can handle.

b) Proceed to step c **if** *Requesti*<= *Availablei*; otherwise, *Pi* will have to wait since the resources are not available

c) Change the state in such a way that the system appears to have given *Pi* the required resources:

Available = Available-Requesti
Allocationi= Allocationi + Requesti
Needi=Needi – Requesti

(run Safety algorithm to check whether safe sequence exists or not)

Content of Need array:

The content of need array is calculated as follows
Need[i][j]=Max[i][j]-Allocation[i][j]

The menu for the program should look like :

1. Accept Allocation
2. Accept Max
3. Calculate Need
4. Accept Available
5. Display Matrices
6. Accept Request and Apply Banker's Algorithm
7. Display the Safe Sequence
8. Exit

Procedural Design – The following is a table of the guidelines for accepting inputs, calculated need and updating available, etc as per Banker's Algorithm and implementation hints.

Procedure	Description	Programming hints
Bankers main	First accept number of processes and number of resources. In a loop print options read an option branch depending on Option	Printf("enter no. of processes & no. of resources ");scanf("%d%d", &n,&m); While (option! = 9) { printoptions(); scanf("%d",&option); switch(option) {case 1: …… }
Accept_matrix (int A[][10])	A generalized procedure which will accept all required matrices, like Allocation, Max depending upon which matrix is passed as parameter	Void accept_matrix(int A[][10]) { int i,j; for (i=0; i<n;i++) for (j=0; j<m;j++) scanf("%d",&A[i][j] }
Accept_available	Accept single dimentional array/vector Available	Void Accept_available() { int I; for (i=0; i<m; i++) scanf("%d" ,&Avail[i]) ; }

Display_matrix	Display Allocation, Max, Need	Void Display_matrix() {…. printf(\n\tAllocation\t\tMax\t\tNeed\n”); for (i=0; i<n;i++) { for (j=0; j<m;j++) printf(“%d”,Allocl[i][j]) ; printf(“\t); for (j=0; j<m;j++) printf(“%d”,Maxil[i][j]) ; printf(“\t); for (j=0; j<m;j++) printf(“%d”,Need[i][j]) ; printf(“\t); } printf(“\n Available\n”); for (j=0; j<m;j++) printf(“%d”,Avail[i][j]) ;printf(“\t); }
Find_Need	Calculate Need matrix	Void Find_Need() { …. for (i=0; i<n;i++) for (j=0; j<m;j++) *Need* [*i,j] = Max*[*i,j*] – *Allocation* [*i,j*]; }
Accept_Request	Accept the request for a process Pi	Void Accept_Request() { int i; printf(“\n enter process no:”); scanf(“%d”,&proc); for (i=0; i<m; i++) scanf(“%d” ,&Requestl[i]) ; }
Bankers_algo	Invoke resource_request_algo and from it Safety_algo	Bankers_algo() { resource_request_algo (); }
Safety-algo	Find if a safe sequence exists	Safety_algo() { int over =0, i, j, k,l=0, flag;

```
  //initialize work = available
for (i=0; i<m; i++)
work[i] = Avail[i];
 while (!over)
{ // check for any not finished process
 for (i=0; i<n; i++)
 {
  if (Finish[i] ==0)
  { flag =0;
    pno=compare_need(i);
     if(pno>-1)break;
  }
 }
 if (i ==n)
 {  printf( "System is unsafe");
    exit(1);
 }
 if (i < n && pno>=0)
{
  for (k=0; k<m; k++)
  work[k] += Alloc[pno][k];
  Finish[pno] = 1;
  Safe[l++] = pno;
 //add process in safe sequence
  if (l >= n) // if all processes over
 {       // print safe sequence
   printf("\n Safe sequence is:");
   for (l=0; l<n; l++)
           printf("P%d\t",Safe[l]);
                   over = 1;
  }
 }
}
}
```

Int compare_need(int p)	Checks if need of processp is <= work or not. If condition true returns process no else returns -1	Int compare_need(int p) { int i,j, flag=0; for (j=0;j< nor; j++) { if (Need[p][j] > work[j]) { flag =1; break; } } if(flag==0) return p;return -1; }
Resource_request_algo	Check if the given requestcan be immediately granted or not	Resource_request_algo() { …// check if Requesti <= Needi for (i=0; i<m; i++) { if Request[i] > Need[proc][i] { printf("error.. process exceeds its Maxdemand"); exit(1); } } // check if Reuest [i] <= Available for (i=0; i<m; i++) { if Request[i] > Avail[i] { printf("Process must wait , resources notavailable"); exit(1); } }// pretend to have allocated requested recources for (i=0; i<m; i++) *{Avail[i] = Avail[i] – Request[i];* *Alloc[proc][i] = Alloc[proc][i] + Request[i]; Need[proc][i] = Need[proc][i] – Request[i]i;* } //run Safety algorithm to check whether safesequence exists or not Safety_algo(); }

Slot I

I) Add the following functionalities in your program
   a) Accept Available
   b) Display Allocation, Max
   c) Display the contents of need matrix
   d) Display Available

Process	Allocation			MAX		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

Slot II

I) Modify above program so as to include the following:
   a) Accept Request for a process
   b) Resource request algorithm
   c) Safety algorithm

Consider a system with ‘n’ processes and ‘m’ resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not.

Assignment Evaluation

0: Not Done	[ ]	1: Incomplete	[ ]	2: Late Complete	[ ]
3: Needs Improvement	[ ]	4: Complete	[ ]	5: Well Done	[ ]

Signature of the Teacher / Instructor _____ Date of Completion _/___/___

Assignment No. 2:

File Allocation Methods

Introduction:

Files are managed by the operating system on secondary storage devices. Hard disk drive is most commonly used storage device.

Physically hard disk is divided into number **sectors** also called as **blocks**. When a file is to be created the free blocks on the disk are allocated to the file.

Operating system maintains the status of each block. **Status of disk block** can be

1) Free
2) Allocated
3) Reserved
4) Bad

Purpose of this practical assignment is to understand various file allocation methods and their implementation.

When user accesses any file, the system has to locate all the disk blocks of that file on disk. Hence operating system has systematic way to maintain, allocate the blocks for each file and releasing the blocks of file when it is deleted.

File Allocation Method is a method that specifies how the blocks allocated to file are maintained so that file can be accessed properly. Most commonly used file allocation methods are as follows:

1) Contiguous File Allocation
2) Linked File Allocation
3) Indexed File Allocation

Data Structures:

1) **Bit Vector:**

   It is used to maintain the free space on disk. It is a vector of size n (where n is number of blocks on disk) with values 0 or 1. These values mark the status of block as whether it is free or allocated. For Ex. It bit vector is: 00111010001101……. It means that first 2 blocks are allocated, block 3,4,5 are free, 6^{th} is allocated, 7^{th} free, $8^{th}$to 10^{th} are not free, 11^{th} and 12^{th} are free and so on.

2) **Directory:**

   Each directory entry maintains the filename along with certain details relevant to file allocation method. Directory list can be maintained statically or dynamically. For Contiguous allocation each directory entry contains filename, starting block number and length. Likewise, for each file allocation method appropriate details are maintained in the directory entry.

Implementation:

- A bit vector (array) of size n which is initialized randomly to 0 or 1.
- A menu with the options
  - Show Bit Vector
  - Create New File
  - Show Directory
  - Delete File
  - Exit
- When Create New File option is selected the program should ask the file name along with number of blocks to allocate to the file. According the free blocks should be searched by using the bit vector and allocated to the file. A new directory entry shall be added in directory list along with appropriate details.
- When Delete File option is selected the program should ask the file name. Release the blocks allocated to the file. Accordingly update the bit vector and remove the entry from directory.

Follow is the program outline for implementing File Allocation methods:

```
#include<stdio.h>
 #include<stdlib.h>
 #include<string.h>
struct File_Details
{
   char File_Name[10];
   int File_start;
   int File_length;
}F1[10];

 int Bit_vector[10],count=0; void initial()
{
   //initialization of bit vector to 1
}
void Display_bit_vector()
{
   //display bit vector
}
int Check_if_free_blocks_available (int initial)
{
   int j=initial;
   int temp = F1[count].File_length;
   while(j<10 && temp!=0)
   {
      //chk free bit vector
     //if yes return 1 else -1
    }
}
void Update_directory (int start, int length)
{
   int i;
   for(i=start;length!=0;i++)
   {
     //assign blocks to the file
    }
}
void Create_File()
{
    int i;
    printf("\nEnter File name : ");
    printf("Enter File length : ");

    // Check_if_free_blocks_available
```

```
    // update directory and print file status
 }
void Display_directory()
{
     //display directory details
}
int File_is_exist_or_not(char temp[])
{
     //chk file details
}
void Delete_File()
{
    printf("\nEnter name to Delete the file : ");
    scanf("%s",name);
    //delete file and no of blocks assigned to it
}
int main()
{
   int choice;
   while(1)
   {  printf("\n1. Show Bit vector \n");
      printf("2. Create New File \n");
      printf("3. Show Directory\n");
      printf("4. Delete File\n");
      printf("5. Exit\n");
      printf("\nEnter choice : ");
      scanf("%d",&choice);
      switch(choice)
      {  case 1: Display_bit_vector(); break;
         case 2: Create_File(); break;
         case 3: Display_directory(); break;
         case 4: Delete_File(); break;
         case 5: exit(0); break;
         default: printf("\nInvalid Choice\n");
      }
    }

}
```

Contiguous File Allocation:

- This file allocation method is also called as Sequential file allocation.
- In this method blocks allocated the file are contiguous. That is if file is of n blocks then all the n blocks are adjacent to one another.
- Advantage of this method is that the file can access sequentially as well as randomly.
- Directory maintains the file name along with starting block number and length (that is number of blocks allocated to file).
- Drawback of this method is that there can be external fragmentation problem and file cannot grow efficiently.

The program outline for the Contiguous File Allocation method is given below, in which includes checking for a continuous block of free memory.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

// Structure to hold file details in the directory
struct file_details
{
    char file_name[10];  // Name of the file (up to 9 characters)
    int file_size;       // Size of the file (in memory blocks)
    int file_start;      // Starting memory block where the file is allocated
} directory[100];          // Array to store up to 100 file details

int bit_vector[100], memory_size, count = 0;  // Bit vector for memory allocation, memory size, and file count
// Initialize the bit vector to represent memory blocks as free
void initialize_bit_vector()
{   int i;
   // Accept total number of memory blocks
   // Initialize all memory blocks as free
  }
// Function to show the current status of memory blocks (bit vector)
void show_memory_blocks()
{   int i;
    printf("Memory Blocks : ");
        // Display the status of all memory blocks (1 = free, 0 = allocated)
 }
```

```
int check_if_freeblock(int block_no)
{   //If the block is free (1)
      return 1;
   else
      return 0;  // If the block is allocated (0)
}
int check_for_continuous_space(int file_size)
{   int i, j;
  // Loop through all blocks to find a continuous free space of the required size
  for(i = 0; i <= memory_size - file_size; i++)
  {    for(_)
       {    // Call check_if_freeblock()  function to check  if any block in this range is
            // free or not
             break;
       }
          // Write a condition to find enough contiguous free space
          {     return i;  // Return the starting block index of the free space  }
   } return -1;  // Return -1 if no continuous space is found
}
// Function to update the bit vector after allocating memory blocks for a file
void update_bit_vector(int start_pos, int file_size)
{   int i;
  // Mark the blocks as allocated (0) in the bit vector
   //For i from start_pos to start_pos + file_size
     {       // Set the blocks as allocated (0)   }
}
void add_directory_entry(char *fileName, int filesize, int start_pos)
{   // Copy the filename to the directory entry
   // Store the file size
  // Store the starting position of the file in the directory
  // Increment the file count
}
void create_file()
{   char filename[10];  // Array to store the file name (up to 9 characters)
    int filesize, start_pos;
    // Read the filename from the user
  // Read the file size from the user
  // Check if there is enough contiguous space for the file
          start_pos = check_for_continuous_space(filesize);
 //Check if no free memory, print an error message
  else
     {      update_bit_vector(start_pos, filesize);
```

```
            add_directory_entry(filename, filesize, start_pos);
           // Print success message
     }
}
// Function to delete a file
void delete_file()
{   int i, j, fileSize, filestart;
   char filename[10];
   // Accept the filename to delete from user
   // Loop through count to search for the file in the directory

  {
         if(strcmp(directory[i].file_name , filename)==0)
        {
              // Get the size of the file to be deleted
             // Get the starting block position of the file
              update_bit_vector(filestart, fileSize);
            // Remove the file from the directory by shifting remaining files
            for(j = i; j < count - 1; j++)
            {         // Shift the file at index (j + 1) to index j    }
                      //Decrement the file count as a file has been removed from the directory
                return;
        }
     }   // If file is not found, print an error message
}
// Function to display the contents of the directory (list of files)
void display_directory()
{   int i;
     // Loop through all the files in the directory and print their details
    //Iterate over each file in the directory (count keeps track of the number of files)
    {   // Print the details of each file: its name, start position in memory, and size      }
}
int main()
{  int choice;
    initialize_bit_vector();  // Initialize the bit vector for memory blocks
    // Menu loop for the user to select options
   // switch case
}
```

Linked File Allocation:

- In this method n blocks file are maintained as chain (linked list) of n blocks. First block of file contains the address or link to second block of file. Second block of file contains the address or link to third of file as so on. Last block of file points to null. That any free block on the storage device can be allocated to the file.
- Advantage of this method is that the file can grow or shrink dynamically and there is no external fragmentation.
- Directory maintains the file name along with starting block number and last block number of a file.
- Drawback of this method is that the file can be access sequentially only. Random file accessing is not possible and wastage of memory for pointer in each block of a file.

The program outline for the Linked File Allocation method is given below:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
struct file_details      // Structure to hold details of files in the directory
{
   char file_name[10]; // Array to store the file name
   int file_size;      // Integer to store the size of the file
   int file_start;     // Start position of the file in memory
   int file_end;       // End position of the file in memory
} directory[100];   // Array to hold up to 100 file details

int bit_vector[100]; // Array to represent memory blocks, -1 indicates free block
int memory_size, free_blocks, count = 0; // Declare memory size, free blocks counter, and file
count
void initialize_bit_vector()    // Function to initialize the bit vector (free blocks)
{
  int i;
   // Prompt user to enter the size of memory blocks
   // Accept the size of memory blocks
   // Initialize the bit vector to -1 (indicating all blocks are free)
   // Iterate the below steps till the memory_size
```

```
    // Initialize each block to free
    // Set free blocks to the total memory size
}
void show_memory_blocks()   // Function to show the current status of memory blocks
{
  int i;
   // Iterate through memory blocks and display their status
   // Display the status of each block
   // Display the count of free blocks
}
// Function to update the bit vector to mark a block as used or free
void update_bit_vector(int start_pos, int value)
{
    // Update the status of the specified block
}
// Function to add a file entry to the directory
void add_directory_entry(char *filename, int filesize, int start_pos, int end_pos)
{
   // Copy the provided filename into the file_name field of the current entry in the directory
  // Set the file_size field of the current entry to the provided filesize
  // Set the file_end field of the current entry to the provided end position
 // Increment the count of files in the directory to prepare for the next entry
}
void create_file()      // Function to create a new file in the memory
{
   int i;
   char filename [10]; // Array to hold file name
   int filesize, start_pos, end_pos, prev_pos, next_pos; // Variables for file size and positions
   // Accept the file name from the user
   // Accept the file size from the user
   // Write a condition to check if there's enough memory to create the file out of all free blocks
   // Otherwise display no enough memory, return
```

```
    //iterate over the below steps, until file size is satisfied
        {Step 1: Generate a random number, between 0 and Maxsize-1;
                  check if bit_vector[random_no] is free;
          Step 2: if free, get the next free block, and set the bit_vector[random_no] =
                  current_freeblock_id;
        } ;
    update_bit_vector (last_block_id, -9); // Mark the end of the file
    // Add the file details (name, size, and start/end positions) to the directory
    // Update the count of free blocks remaining after file creation
    // Print a message indicating the file has been created with its details
 }
void delete_file ()  // Function to delete a file from memory
{
  int i, j, start_pos, next_pos, temp; // Variables for file deletion
  char filename [10];

  //Accept the file name to delete from user
  //Iterate through the directory to find the file
     // Check if the current file in the directory matches the input filename
        // Get the starting position of the file from the directory entry
        // Initialize next_pos to the starting position of the file

       // Iterate a loop to free each block until the end of the file
          // Continue until the end marker is reached
         // Hold the current block position in a temporary variable
         // Get the next block allocated to this file from the bit vector
         // Mark the current block as free in the bit vector
         // Increment the count of free blocks available

       // Move files down in the directory to fill the gap left by the deleted file
         // Shift entries to the left to overwrite the deleted file

       // Decrement the total count of files in the directory
         // Display the message indicating the file has been deleted
    }
void display_directory()     // Function to display the file directory
{
  int i; // Loop index variable to iterate through the directory
    // Display the header for the directory listing
    // Loop through the directory to show details of each file
```

```
      // Iterate over each entry in the directory
      // Display the details of the file at the current index in the directory (File name, size)
}
// Main function to drive the program
int main ()
{
  int choice; // Variable to hold user menu choice
  initialize_bit_vector(); // Initialize bit vector to set up memory
  // Menu-driven loop
    // Continue until user chooses to exit
}
```

Index File Allocation:

- This method overcomes the drawbacks of Sequential File Allocation method as well as Linked Allocation Method.
- In this method a special Index Block is maintained for each file. This block contains the list of block numbers allocated to the file. When file is to be accessed the index block will give the list of blocks allocated to that file.
- Hence in this method file can be accessed sequentially as well as randomly.
- Each available free block on disk can be allocated to any file on demand and that number is added to the index block of that file.
- Directory maintains the file name along with its index block number and number of entries in it.
- Drawback of this method is the overhead of maintaining index block.

The program outline for the Indexed File Allocation method is given below:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_FILES 1000  // Maximum number of files that can be stored
#define MAX_BLOCKS 1000  // Maximum number of memory blocks available

typedef struct
{
  char *filename;  // Name of the file
  int file_size;  // Size of the file (in blocks)
  int index_block;  // Block no of the index block in the memory
```

```
} file_detail;
// Structure to represent memory blocks

typedef struct
{
   int allocated;  // Whether the block is allocated (1 = allocated, 0 = free)
   int index[MAX_BLOCKS - 1];  // Array to store block indexes of the file
} block;

 block bit_vector[MAX_BLOCKS];       // Global bit vector for memory allocation tracking
 int memory_size = 0;        // Total memory size
 int free_memory;            // Available free memory
 file_detail *dir[MAX_FILES];    // Directory for storing file details

void initialize_bit_vector()
{
   int i;
   // Accept  total memory size from user
   // Initialize all memory blocks as unallocated (0)
   // Initialize the file directory as empty
 }
void show_memory_blocks()
{
   int i;
  // Display the  bit vector (allocation status of each block)
 }
void update_directory(file_detail *file)
{
   int i;
   // Find an empty slot in the directory to store the file
   For i from 0 to MAX_FILES
   {
      // Check if the current directory entry is empty (i.e., NULL)
      {     // Store the file details at the first available empty slot
            return;
      }
   }
}
int get_file(char *filename)
{   int i;
   // Search for the file in the directory
    For i from 0 to MAX_FILES
       // use condition to check if the directory entry is not NULL and if the file
```

```
       // name matches(use strcmp() function to match the filename
              // Return the index of the file if found
            // If no matching file is found, return -1 to indicate the file is not present in the directory
 }
void create_file()
{    char filename[256] = { 0 };
    file_detail *file = NULL;
    int i, file_size, index_block, id;

   // Accept the filename from a user
   // Display an error message if the file already exists
      return;
   // Accept the file size from the user
   // Check if there is enough free memory to allocate the file
    {     // Display  an error message if there isn't enough free memory
          return;
     }
       // Allocate memory for the file structure using malloc function
       // Copy the filename string into the newly allocated memory
            // Set the file's size in the file structure
     // Set the index block to -1 initially
              // Generate a random block ID within the memory size
       if (file->index_block == -1)
       {         // Mark the  Bit_vector[id] as allocated
                // Set the current block ('id') as the file's index block
       }
       else  // file already has an index block
      {       // Link the new block ('id') to the file's index block by adding it to the 'index' array of
the index block
            bit_vector[_______].index[i] = id;
            bit_vector[id].allocated = ___;  // Mark the  Bit_vector[id] as allocated
      }
        // Reduce free memory after allocation
          // call  update_directory() function
  }
void delete_file()
{
    char filename[256] = { 0 };
    file_detail *file;
    int i, idx, id;
    //Accept filename from the user
    // Find the file by name (call get_file() function)
    // Check if the index is -1, which typically indicates that the file was not found
```

```
    {
      // Print an error message indicating the file was not found
       return;
     }
     // Retrieve the file from the directory using the index 'idx'
     // Free the blocks allocated to the file
     // Loop to iterate till file_size
      {     // Get the block ID from the index array of the file's index block
               id = bit_vector[file->index_block].index[i];
           // Mark the block as free by setting its 'allocated' status to 0 in the bit vector
               bit_vector[id].allocated = __;
       }
    // Free the index block by marking it as unallocated in the bit vector
               bit_vector[file->index_block].allocated = ____;
    // Increase the free memory by the file's size, making the space previously occupied by the file
available
   // Remove the file entry from the directory by setting its position to NULL (file no longer exists in
the directory)
  // Free the memory that was allocated for the filename string
  // Free the memory allocated for the file structure itself, releasing the resources
  // associated with the file
  // Print a message indicating that the file has been successfully deleted
 }
 void display_dir()
 {    int i, j;
      file_detail *file;
      // display  index,  size,  name,  blocks
      // Loop to iterate for all MAX_FILES
       {
         // Condition to check
               continue;     // skip if found  empty directory entries
               file = dir[i];
        // display index_block, file_size, filename
        // Print the blocks allocated to the file
       }
 }
 int main()
 {      int choice;
        initialize_bit_vector();  // Initialize the memory system
        // Accept choice from user using switch case
 }
```

Slot - I

Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

Slot - II

Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options as mentioned above and implement each option.

Slot- III

Write a program to simulate Indexed file allocation method. Assume disk with n number of blocks. Give value of n as input. Write menu driver program with menu options asmentioned above and implement each option.

Assignment Evaluation

0: Not Done	[ ]	1: Incomplete	[ ]	2: Late Complete	[ ]
3: Needs Improvement	[ ]	4: Complete	[ ]	5: Well Done	[ ]

Signature of the Instructor:_____ Date of Completion____/____/____

Assignment No. 3:

Disk Scheduling Algorithms

Introduction:

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, meeting this responsibility entails having fast access time and
large disk bandwidth. Both the access time and the bandwidth can be improved by managing the order in which disk I/O requests are serviced which is called as disk scheduling. The simplest form of disk scheduling is, of course, the first - come, first - served (FCFS) algorithm. This algorithm is intrinsically fair, but it generally does not provide the fastest service. In the SCAN algorithm, the disk arm starts at one end, and moves towards the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk. At the other end, the direction ofhead movement is reversed, and servicing continues. The head continuously scans back and forthacross the disk. C - SCAN is a variant of SCAN designed to provide a more uniform wait time.
Like SCAN, C - SCAN moves the head from one end of the disk to the other, servicing requests along the way. When the head reaches the other end, however, it immediately returns to the beginning of the disk without servicing any requests on the return trip

DISK SCHEDULING ALGORITHM

- The algorithms used for disk scheduling are called as disk scheduling algorithms.
- The purpose of disk scheduling algorithms is to reduce the total seek time.

First Come First Serve (FCFS)

FCFS is the simplest disk scheduling algorithm. As the name suggests, this algorithm entertains requests in the order they arrive in the disk queue. The algorithm looks very fair and there is no starvation (all requests are serviced sequentially) but generally, it does not provide the fastest service.

Advantages-

- It is simple, easy to understand and implement.
- It does not cause starvation to any request.

Disadvantages-

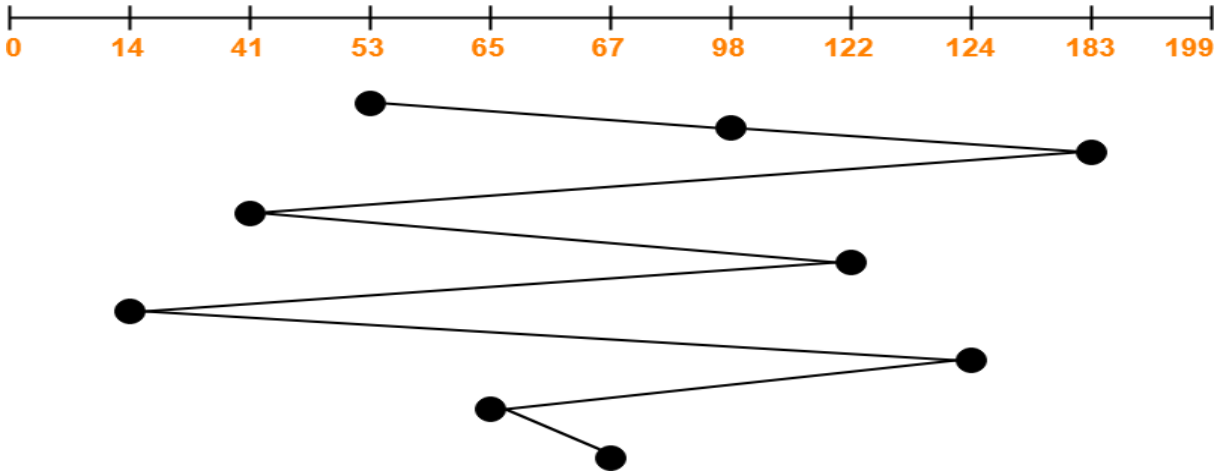
- It results in increased total seek time.
- It is inefficient

Program outline for implementing FCFS :

```
#include <stdio.h>
#include <stdlib.h>
void accept_requests(int requests[], int nreqs)
{    int i;
     // Accept the requests from user
    // Iterate the below step nreqs times
    // Read the request value into request [] array.
 }
 int fcfs(int head, int requests[], int nreqs)
 {
    int i, total_movement = 0;
    printf("Sequence: ");
       For i from 0 to nreqs - 1
            // Print head
              //Calculate Total Head movement
            head = requests[i]
       End For
    // Print Head and Total Head movement
  }
int main()
{
    int requests[200];
    int nreqs, head;
   // Accept No of requests from user
    accept_requests(requests, nreqs);
   //Accept Initial head position
    fcfs(head, requests, nreqs);
}
   return 0;
}
```

Example

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The FCFS scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is__.



Total head movements incurred while servicing these requests

= (98 – 53) + (183 – 98) + (183 – 41) + (122 – 41) + (122 – 14) + (124 – 14) + (124 – 65) + (67– 65)

= 45 + 85 + 142 + 81 + 108 + 110 + 59 + 2

= 632

SSTF Disk Scheduling Algorithm-

SSTF stands for **Shortest Seek Time First**.

- This algorithm services that request next which requires least number of head movements from its current position regardless of the direction.
- It breaks the tie in the direction of head movement

Advantages-

It reduces the total seek time as compared to FCFS.

- It provides increased throughput.
- It provides less average response time and waiting time.

Disadvantages-

There is an overhead of finding out the closest request.

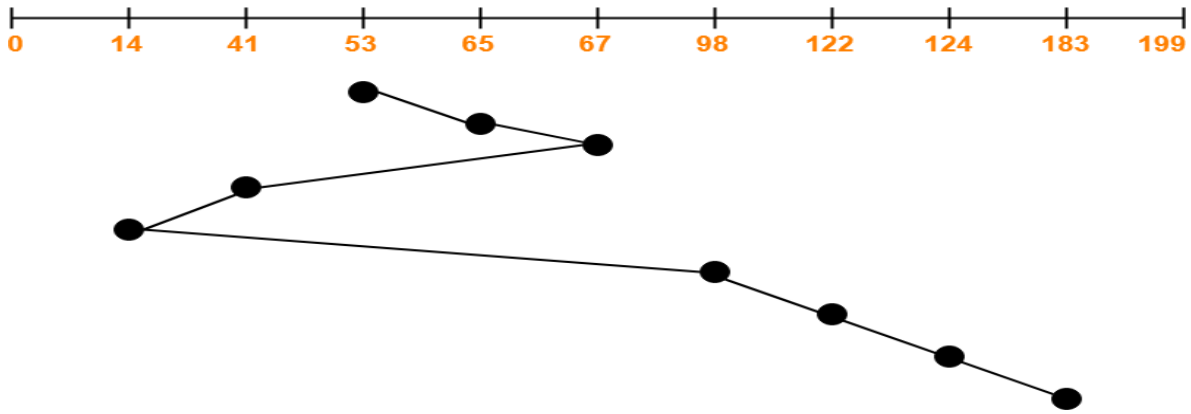
- The requests which are far from the head might starve for the CPU.
- It provides high variance in response time and waiting time.
- Switching the direction of head frequently slows down the algorithm.

Program outline for implementing SSTF:

```
#include <stdio.h>
#include <stdlib.h>

void accept_requests(int requests[], int nreqs)
{
   int i;
   //Accept disk request
   sort(requests, nreqs);
}
int sort(int arr[], int n)
{    //    Sort the requests using any sorting algorithm
}
int sstf(int head, int requests[], int nreqs)
{
   int i, total_movement = 0;
   int visited[200] = { 0 };
   int nearest, count = nreqs;
   int left, right, dist;
   int left_dist, right_dist, direction = 1;

   while (count-- > 0)     // display the request greater than or equal to the head by using loop :
   {   // Loop to process all the requests
       // Calculate the difference in distance between the left and right request from the current head
       if (left == -1)
       {
            // If no unvisited request is found on the left side, move to the right
       }
       else if (right == nreqs)
       {
          // If no unvisited request is found on the right side, move to the left
       }
      // If the left request is closer or if the direction is left and distances are equal, goto left direction
      // Otherwise goto right direction
      // Update total head movement by adding the distance between the head and the nearest request
      // Move the head to the nearest request and mark the current request as visited
      // Print the head position after servicing the request
   }
        // Print the total head movement after all requests are serviced
 }
```

```
int main()
 {
   int requests[200];
   int nreqs, head;
   // Accept the number of requests
          accept_requests(requests, nreqs);
    // Accept the initial head position
    // Call the SSTF function to perform the disk scheduling and print the results
    return 0;
}
```

Example



Total head movements incurred while servicing these requests:

= (65 – 53) + (67 – 65) + (67 – 41) + (41 – 14) + (98 – 14) + (122 – 98) + (124 – 122) + (183 –124)

= 12 + 2 + 26 + 27 + 84 + 24 + 2 + 59

= 236

SCAN DISK SCHEDULING ALGORITHM

Given an array of disk track numbers and initial head position, our aim is to find the total number of seek operations done to access all the requested tracks if SCAN disk scheduling algorithm is used.

SCAN (Elevator) algorithm

In SCAN disk scheduling algorithm, head starts from one end of the disk and moves towards the other end, servicing requests in between one by one and reach the other end. Then the direction

of the head is reversed and the process continues as head continuously scan back and forth to access the disk. So, this algorithm works as an elevator and hence also known as the elevator algorithm. As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

Advantages of SCAN (Elevator) algorithm

- This algorithm is simple and easy to understand.
- SCAN algorithm has no starvation. 3. This algorithm is better than FCFS Scheduling algorithm.

Disadvantages of SCAN (Elevator) algorithm

- More complex algorithm to implement.
- This algorithm is not fair because it cause long waiting time for the cylinders just visited by the head.
- It causes the head to move till the end of the disk in this way the requests arriving ahead of the arm position would get immediate service but some other requests that arrive behind the arm position will have to wait for the request to complete.

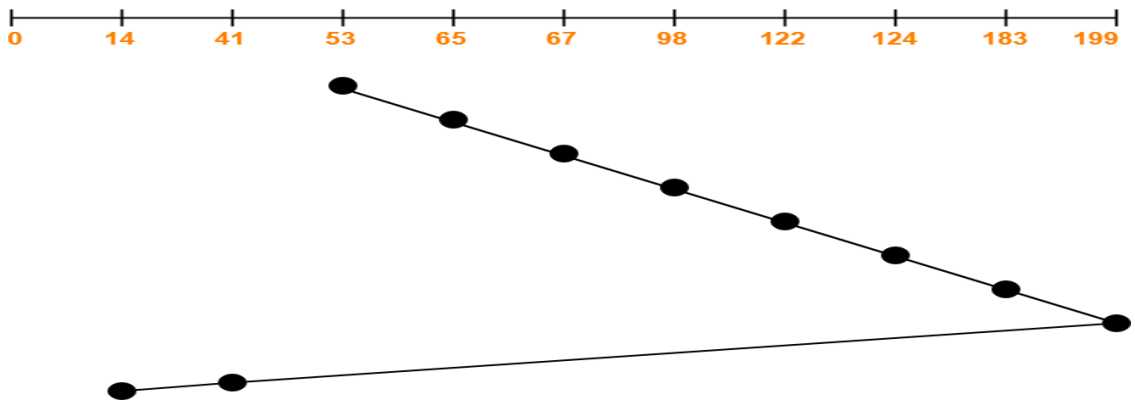
Program outline for implementing SCAN :

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_CYL 200

void sort(int arr[], int n)
{
    int i, j, tmp;
   // Sort all the request using any sorting technique
  }
void accept_requests(int requests[], int nreqs) // Function to accept disk requests from the user
{
    int i;
    // Prompt user to enter requests
    // Loop to read each request from user
    // Store each request in the array
    sort(requests, nreqs); // Sort the requests in ascending order
}
int scan(int head, int direction, int requests[], int nreqs)
{
    int i;
    int total_movement = 0;
    int count = nreqs;
    int visited[MAX_CYL] = {0};
   // Condition to Find the index of the first request that is greater than or equal to the current head
position
        // Check if current request is greater than or equal to head
            break;
         // Print initial head position
  // Infinite loop to process requests in the current direction
     {
         // Skip already visited requests
                continue;
        // Calculate total movement by the distance to the current request
            head = requests[i];    // Move head to the current request
            visited[i] = 1;     // Mark this request as visited
       // Print the current head position
       // Decrease the count of remaining requests
           // If all requests are processed
            break;

        // If we reach the beginning (0)
            {
                  // Add head position to total movement
                 // Change direction
                // Move head to the start
               // Print the new head position
            }
```

```
      // else If we reach the last request
            {
                 // Update total movement to the end
                // Change direction
               // Move head to the last cylinder
              // Print the new head position
            }
     }
   // Print total head movement
}
int main()
{
   int requests[200];
   int nreqs, head, direction;

    // Prompt user for the number of requests
    // Read number of requests

   accept_requests(requests, nreqs); // Accept and sort the requests

    // Prompt user for the initial head position
    // Read initial head position

    // Prompt user for the direction of movement
     // Read the direction (1 for right, -1 for left)

   scan(head, direction, requests, nreqs);
   return 0;
}
```

Example

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65,67. The SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is___.



Total head movements incurred while servicing these requests

= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (199 – 183) + (199 – 41) + (41 – 14)

=12 + 2 + 31 + 24 + 2 + 59 + 16 + 158 + 27 = 331

C-SCAN DISK SCHEDULING ALGORITHM

Description:

- Circular-SCAN Algorithm is an improved version of the SCAN Algorithm.
- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction.
- It then returns to the starting end without servicing any request in between.
- The same process repeats.

Advantages-

- The waiting time for the cylinders just visited by the head is reduced as compared to the SCAN Algorithm.
- It provides uniform waiting time.
- It provides better response time.

Disadvantages-

- It causes more seek movements as compared to SCAN Algorithm.
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

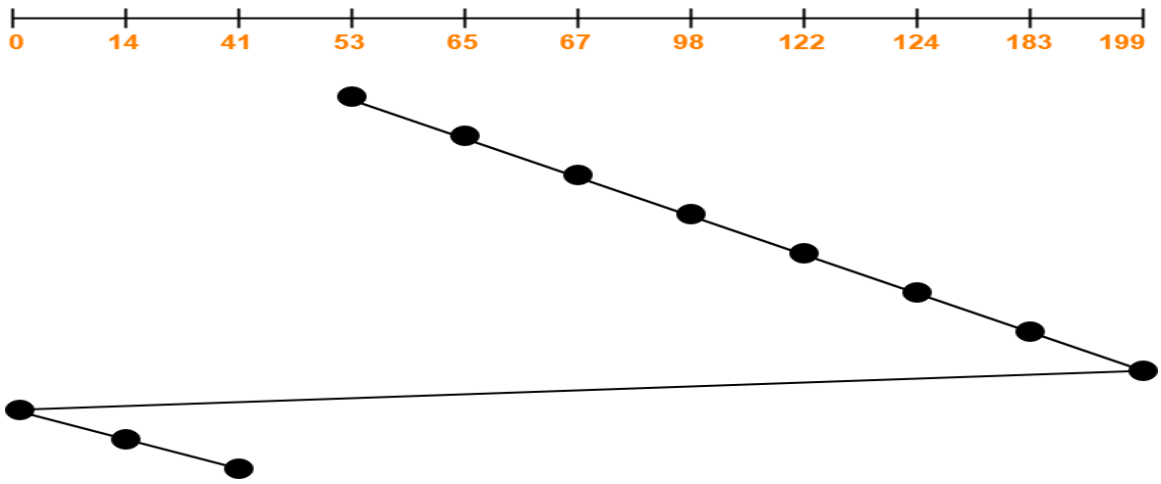
Program outline for implementing C-SCAN :

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_CYL 200

void sort(int arr[], int n)   // Function to sort the request array using bubble sort
{    int i, j, tmp;
   // Use sorting algorithm to sort the request array in ascending order
}
// Function to accept disk requests from the user and sort them
void accept_requests(int requests[], int nreqs)
{    int i;
   // Accept the disk requests from user
            sort (requests, nreqs);
}
int c_scan(int head, int direction, int requests[], int nreqs)
{  int i;
   int traversed = 0;  // Flag to indicate whether we have traversed the entire disk
   int total_movement = 0;  // Variable to track total head movement
   int count = nreqs;  // Counter to track remaining requests to be served
   int visited[MAX_CYL] = {0};  // Array to mark visited requests

   // Find the first request that is greater than or equal to the current head position
   // Check if the current request is greater than or equal to the current head position

   // Calculate the movement from current head to the request
   // Calculate the absolute distance moved from the current head position to the requested cylinder
      // Display the current request's position served by the head
     // Decrease the remaining requests counter, and break the loop when no requests are left
      if (--count <= 0)
         break;
      if (traversed) // If we have already traversed the entire disk, continue processing
         continue;
      if (i == 0)       // If we reach the beginning of the disk (cylinder 0), reverse direction
      {   // Add movement to cylinder 0 and then to the last cylinder
         // Move the head to the last cylinder
         // Mark as 1 that we've traversed the disk
         // Print cylinder 0 and the last cylinder
      }
     else if (i =__)   // If we reach the last request, reverse direction
      {      // Add movement from head to the last cylinder and then back to cylinder 0
           // Move the head to the first cylinder
           // Mark as 1 that we've traversed the disk
          // Print the last cylinder and cylinder 0
      }
    }  // display the total head movement after serving all requests
 }
```

```
int main ()
{    int requests[200];  // Array to store disk requests
   int nreqs, head, direction;
   accept_requests(requests, nreqs); // Accept number of disk requests from user
   // Accept the initial head position from user
   // Accept the direction of the head movement from user/ 1 for right, -1 for left
    c_scan(head, direction, requests, nreqs);
   return 0;
}
```

Example:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The C-SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is_____.



Total head movements incurred while servicing these requests

= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (199 – 183) +(199 – 0) + (14 – 0) + (41 – 14)

= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 27

= 386

Slot-I

i. Write an OS program to implement FCFS Disk Scheduling algorithm.

ii. Write an OS program to implement SSTF algorithm Disk Scheduling algorithm.

Slot-II

i. Write an OS program to implement SCAN Disk Scheduling algorithm.

ii. Write an OS program to implement C-SCAN algorithm Disk Scheduling algorithm.

Assignment Evaluation

0: Not Done	[ ]	1: Incomplete	[ ]	2: Late Complete	[ ]
3: Needs Improvement	[ ]	4: Complete	[ ]	5: Well Done	[ ]

Signature of the Instructor:_____Date of Completion_____/_____/_____

Assignment 4:
A Case Study on Mobile Operating System

Introduction:

A mobile operating system controls everything from handling the input, to controlling the memory and the overall functioning of the device. It also manages the communication and the interplay between the mobile device and other compatible hardware such as computers, televisions or printers. In short mobile operating system does computing based on mobility.

Mobile Operating System:

Mobile computing is an ability to compute remotely while on the move. We can also say that computing on mobility is called mobile computing. It is a kind of technology which allows transmission of any kind of data (voice, graphics, binary or else) using wireless device. Mobile computing consists of three components:

1. **Wireless Communication Technology**

The wireless communication technology refers to the infrastructure that provides a faultless and reliable communication facility. The technology consists of wireless protocols, services, bandwidth and other necessary components which are required by the wireless communication.

2. **Mobile Hardware**

Any device that can be easily carried and moved from one place to another is called a mobile device / mobile hardware. It may be laptop, PDA, Tablet, smart phone etc. These kinds of devices have a capability of sending and receiving signals. These devices can also transmit and receive signals at the same time.

3. **Mobile Software**

Mobile software is the program that runs on the mobile hardware. It is responsible to manage the hardware. It also deals with the requirement of mobile applications. It is an interface between mobile applications and mobile hardware.

Generally, mobile software is called mobile operating system (MOS). Mobile operating system is the kernel of the mobile hardware.

Android, iOS, Window Phone, BlackBerry are the example of mobile operating systems.

SLOT 1

Q1. Write a case study on a Mobile operating system for detailed analysis covering the OS's history, features, and future prospects. Select any one Mobile Operating System from Palm OS, Symbian OS or Blackberry Operating System and explain the following points:

- Introduction of Operating System
- Architecture and Features of Operating System
- The evolution of mobile operating systems.
- Future trends in Mobile Operating Systems.
- Advantages and Disadvantages

Assignment Evaluation

0: Not Done	[ ]	1: Incomplete	[ ]	2: Late Complete	[ ]
3: Needs Improvement	[ ]	4: Complete	[ ]	5: Well Done	[ ]

Signature of the Teacher/ Instructor_____ Date of Completion_/___/___